

LE_AUDIO 应用设计说明

珠海市杰理科技股份有限公司

Zhuhai Jieli Technology Co.,LTD

版权所有，未经许可，禁止外传

修改记录

版本	更新日期	描述
V1.0	2022/12/15	编写初版文档
V1.1	2022/12/26	修改部分接口及参数描述，增加音频流程描述
V1.2	2023/1/12	添加常见问题解决方法
V1.3	2023/4/7	删除函数接口描述



目录

一、 名词解析	1
二、 BIG	2
(一) 代码结构介绍	2
1. 目录介绍	2
2. 文件介绍	3
3. 各个文件之间的调用关系	4
4. 启动和关闭流程	4
(二) 参数及配置说明	8
1. board_jl701n_xxx.cfg.h	8
2. big_params.c	9
3. broadcast.c	9
4. 其他文件	9
(三) 发送和接收状态及参数同步	10
三、 CIG	13
(一) 代码结构介绍	13
1. 目录介绍	13
2. 文件介绍	14
3. 各个文件之间的调用关系	15
4. 启动和关闭流程	15
(二) 参数及配置说明	18
1. board_jl701n_xxx_cfg.h	18
2. cig_params.c	19
3. connected.c	19
4. 其他文件	20
(三) ACL 数据交换	20
1. 任务同步执行	20
2. 按键消息同步	21
3. 互发数据给对方	21
四、 音频流程介绍	22
(一) 流程介绍	22
(二) BIG 和 CIG 音频流程算法接入	22

五、 常见问题及解决方法	23
(一) 发送 ACL 数据异常	23
1. Note: cis perip acl tx hdl(0x50) > 9	23



一、名词解析

以下将会对本文档所使用的名词进行介绍。

名词	全称	解析
BIG	Broadcast Isochronous Group	基于广播的无线音频传输所使用的一种 BLE 单向传输协议。
BIS	Broadcast Isochronous Stream	与 BIG 是从属关系，一个 BIG 下可以有多个 BIS, 每个 BIS 可以广播一路音频。
CIG	Connected Isochronous Group	基于连接的无线音频传输所使用的一种 BLE 双向传输协议。
CIS	Connected Isochronous Stream	与 CIG 是从属关系，一个 CIG 下只能有两个 CIS, 每个 CIS 可以传输一路音频。
Broadcast		取 BIG 第一个单词。
Connected		取 CIG 第一个单词。
音源模式		指 BT/MUSIC/AUX/PC 等模式

二、BIG

（一）代码结构介绍

1. 目录介绍

基于 BIG 的无线音频传输功能由以下几个文件组成：

蓝牙相关

- （1）wireless_dev_manager.c
- （2）wireless_dev_manager.h
- （3）big.c
- （4）big.h

音频相关

- （1）broadcast_audio_sync.c
- （2）broadcast_audio_sync.h
- （3）broadcast_dec.c
- （4）broadcast_dec.h
- （5）broadcast_enc.c
- （6）broadcast_enc.h
- （7）broadcast_codec.h

应用层相关

- （1）app_broadcast.c
- （2）app_broadcast.h
- （3）broadcast.c

- (4) broadcast_api.h
- (5) big_params.c
- (6) wireless_params.h

2. 文件介绍

由于蓝牙相关文件改动较少，此处不做深入介绍。

应用层的代码主要由六个文件组成，还有部分代码穿插在其他文件的应用流程里面，此处重点介绍主要的六个文件。

(1) app_broadcast.c

该文件存放了应用流程二次开发所需的 API，BIG 连接等相关事件获取的 API。

(2) app_broadcast.h

该文件为为 app_braodcast.c 的头文件，存放了相关的结构体定义、函数声明。

(3) broadcast.c

该文件进行二次开发时改动较少。其代码内容主要是耦合蓝牙和音频相关接口，并把基础流程封成 API 给 app_broadcast.c 使用。

(4) broadcast_api.h

该文件为为 broadcast.c 的头文件，存放了相关的结构体定义、函数声明。

(5) big_params.c

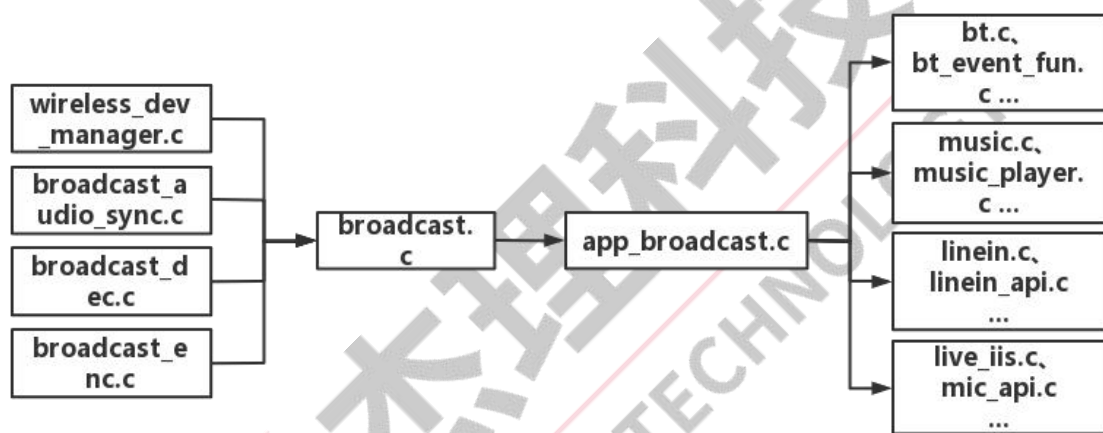
该文件主要存放 BIG 及音频编解码所需的参数以及设置参数的 API。每个模式对应一个参数表，在设置参数时会根据当前模式使用对应的参数表。

另外接收端音频编解码参数支持每次开启 BIG 时自动适配发送端的参数，保证发送端接收端参数一致。BIG 工作过程中暂不支持动态修改参数。

(6) wireless_params.h

该文件为为 big_params.c 的头文件，存放了相关的结构体定义、函数声明。

3. 各个文件之间的调用关系



4. 启动和关闭流程

(1) 带蓝牙后台情况下，开机后应先进蓝牙模式，再切换到其他模式，因为进入蓝牙模式时会自动初始化蓝牙协议栈；如果直接进入其他模式，则需手动初始化协议栈。

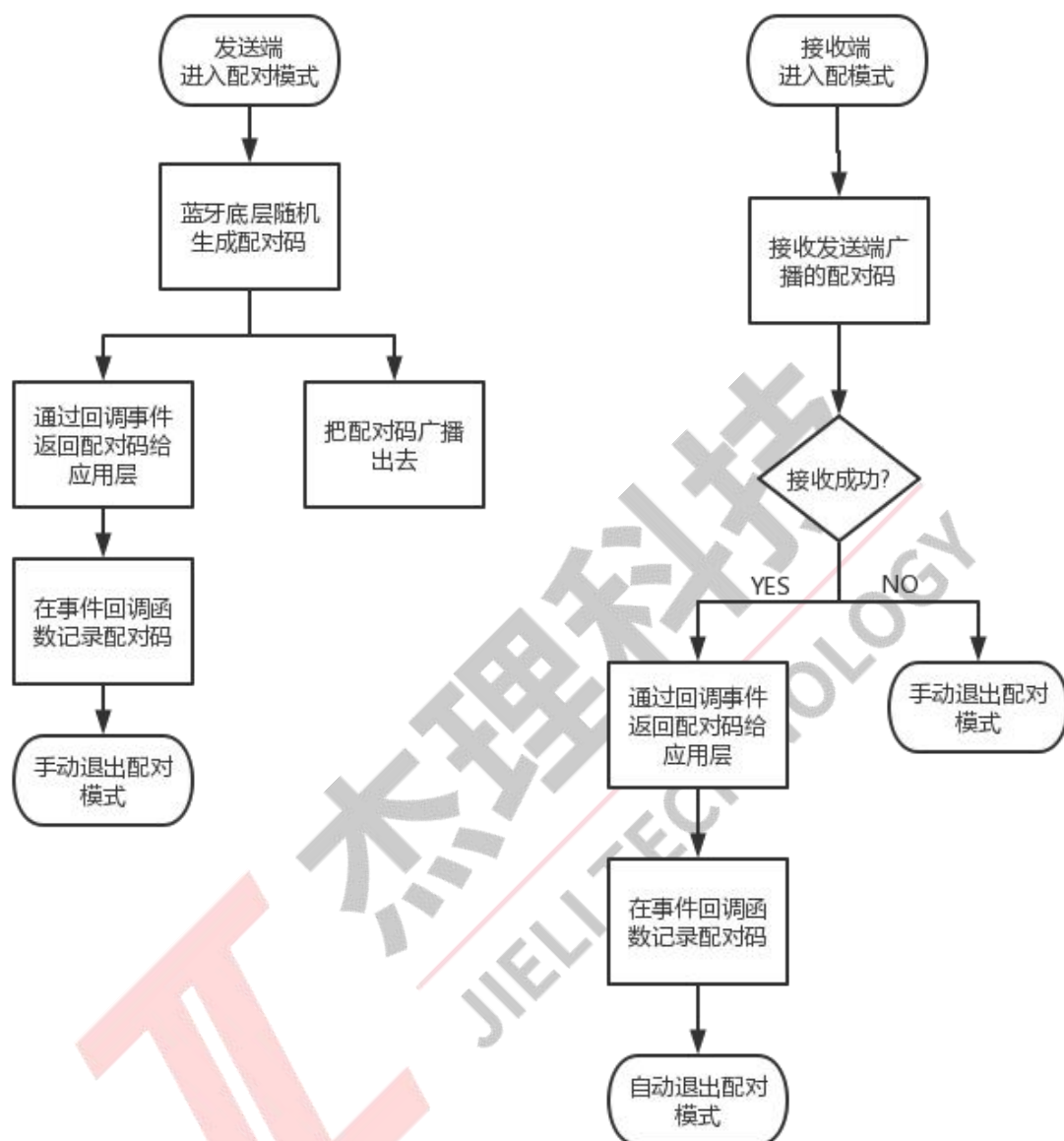
(2) 不带后台情况下，由于退出蓝牙模式时蓝牙协议栈也会被关闭，因此切换到其他模式后需手动初始化蓝牙协议栈，退出模式时也要手动关闭蓝牙协议栈。

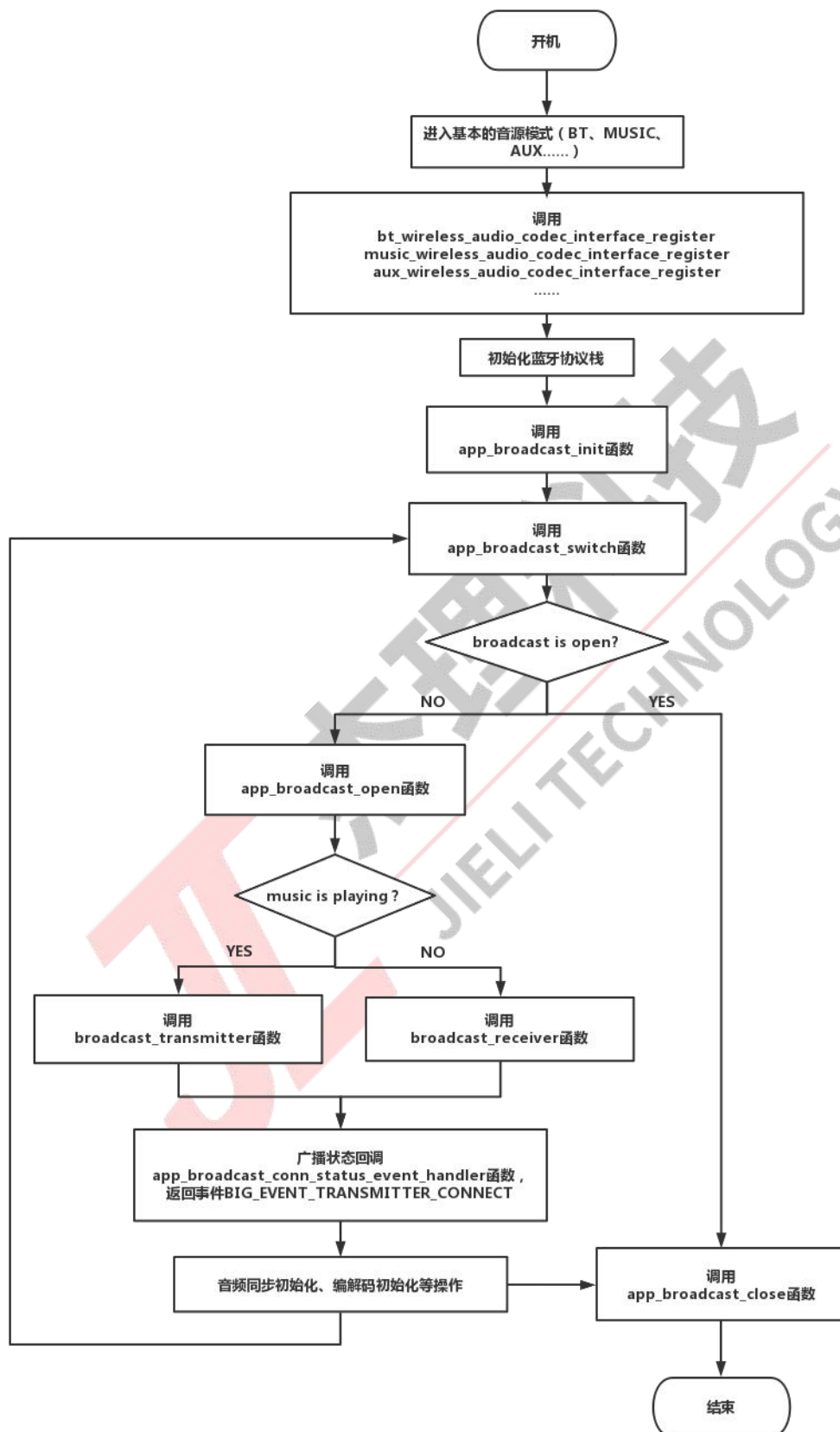
(3) 手动初始化蓝牙协议栈可以调用 `btstack_init_in_other_mode` 函数，退出则调用 `btstack_exit_in_other_mode` 函数。

(4) 由于 BIG 广播时所运行的音频流程与非广播不一样，所以需要注册音频流程切换接口，可通过 `xxx_codec_interface_register` 来注册。

(5) SDK 默认在 BIG 广播过程中播放音乐和暂停音乐会自动切换广播角色。播放音乐时会切换为发送端；暂停音乐时会切换为接收端。由每个模式暂停和播放的位置调用 `app_broadcast_deal` 函数实现。

(6) `broadcast` 支持使用配对码进行配对功能，目的防止交叉配对。使用配对码配对功能需先进入一个单独的配对模式，该模式不能与 BIG 功能同时开启。在配对模式下，发送端会对外广播四个字节的配对码，接收范围内的接收端收到后记录配对码到 VM，并自动退出配对模式（发送端需手动退出）。发送端的配对码由蓝牙底层生成并通过回调事件返回给应用层，应用层再把配对码记录到 VM。当下次开启 BIG 功能时，会先从 VM 读取配对码并配置给蓝牙底层；此时只有发送端和接收端的配对码一致时，接收端才会监听发送端设备的数据。当配置给蓝牙底层的配对码为 0 时，接收端不做设备过滤，相对于可以监听任意设备。





(二) 参数及配置说明

1. board_jl701n_xxx.cfg.h

参数名/配置名	含义	默认值
TCFG_BROADCAST_ENABLE	BIG 总开关	0
LEA_BIG_CTRLER_TX_EN	BIG 发送端开关	1
LEA_BIG_CTRLER_RX_EN	BIG 接收端开关	1
BROADCAST_DATA_SYNC_EN	广播状态同步功能开关	1
BROADCAST_LOCAL_DECODE_EN	广播发送端本地播放开关	1
BROADCAST_RECEIVER_CLOSE_EDR_CONN	广播从机关闭经典蓝牙可发现可连接	0
JLA_CODING_SAMPLERATE	JLA 编解码采样率	48000 (暂不支持 44100)
JLA_CODING_FRAME_LENGTH	一帧编码的 pcm 时间长,单位 ms	100 (只支持 25, 50, 100)
JLA_CODING_CHANNEL	JLA 的通道数, 其他参数不变, 实际单声道的码率是双声道码率的 2 倍	2
JLA_CODING_BIT_RATE	JLA 的编解码码率	JLA_CODING_CHANNEL 为 1 时:64000 JLA_CODING_CHANNEL 为 2 时:96000
TCFG_LIVE_AUDIO_LOW_LATENCY_EN	实时音频低延时使能, BIG 和 CIG 两种模式均可使用	0

2. big_params.c

参数名/配置名	含义	默认值
BIG_PAIR_NAME	配对名	"br28_big_soundbox"
TX_USED_BIS_NUM	BIS 发送通道数	1
RX_USED_BIS_NUM	BIS 接收通道数	1
XXX_SDU_PERIOD_MS	蓝牙发包时间间隔	20
big_xxx_codec_params	不同音源模式对应的 JLA 编解码参数	见代码
big_xxx_tx_param	不同音源模式对应的 BIG 发送参数。其中.rtn 为重复次数，.mtl 为预发包数量。rtn 和 mtl 都会影响接收端收包质量和距离，两个参数越大，BIG 距离越远，经典蓝牙距离越短。另外，增大 mtl 还是增大延时。	见代码
big_rx_param	BIG 接收参数。其中.rx.bis 为接收 bis 通道的序号（序号从 1 开始）	见代码

3. broadcast.c

参数名/配置名	含义	默认值
BROADCAST_DEC_OUTPUT_CHANNEL	选择解码器解码输出的声道	AUDIO_CH_LR

4. 其他文件

参数名/配置名	含义	默认值
xxx_audio_process_switch_func	配置普通音源模式音频流程和 BIG 音频流程切换接口，本地音频播放和暂停状态获取接口	见代码

（三）发送和接收状态及参数同步

广播音箱在广播期间会进行周期广播，广播包里面可以包含需要同步的状态数据。目前 SDK 已经实现音量同步和关机同步的操作，其他操作客户可自行扩展。目前广播周期为 120ms，可通过 big_params.c 里面 big_xxx_tx_param 参数进行修改。相关流程和代码如下：

位置	big_params.c
说明	修改状态数据广播周期.padv_init_ms，必须为 sdu_period_ms 的倍数
代码	<pre> 1. static big_parameter_t big_bt_tx_param = { 2. .pair_name = BIG_PAIR_NAME, 3. .cb = &big_tx_cb, 4. .num_bis = TX_USED_BIS_NUM, 5. .ext_phy = 1, 6. .enc = 0, 7. .bc = BIG_BROADCAST_CODE, 8. #if defined(WIRELESS_1tN_EN) && (WIRELESS_1tN_EN) 9. .form = 1, 10. #else 11. .form = 0, 12. #endif 13. 14. .tx = { 15. .phy = BIT(1), 16. .aux_phy = 2, 17. .padv_init_ms = 120, 18. }, 19. }; </pre>

位置	broadcast_api.h
说明	在“//解码参数同步”上面添加需要同步的状态变量
代码	<pre> 1. /*! \brief 广播同步的参数 */ 2. struct broadcast_sync_info { 3. // 状态同步 4. u8 volume; 5. u16 softoff; 6. // 解码参数同步 7. u8 sound_input; 8. u8 nch; 9. u16 frame_size; 10. int coding_type; 11. int sample_rate; 12. int bit_rate; 13. }; </pre>

位置	app_broadcast.h
说明	在此枚举结构中添加需要同步的状态枚举
代码	<pre> 1. enum { 2. BROADCAST_SYNC_VOL, 3. BROADCAST_SYNC_SOFT_OFF, 4. }; </pre>

位置	app_broadcast.c
说明	使用该接口更新广播包数据
代码	<pre> 1. int update_broadcast_sync_data(u8 type, int value) </pre>
例子	<pre> 1. update_broadcast_sync_data(BROADCAST_SYNC_VOL, volume); </pre>

位置	app_broadcast.c
说明	该函数实现接收端接收到广播包后解析数据，并进行状态同步

代码	1. <code>int</code> broadcast_padv_data_deal(<code>void</code> *priv)
----	--



三、CIG

（一）代码结构介绍

1. 目录介绍

基于 CIG 的无线音频传输功能由以下几个文件组成：

蓝牙相关

- （1）wireless_dev_manager.c
- （2）wireless_dev_manager.h
- （3）cig.c
- （4）cig.h

音频相关

- （1）broadcast_audio_sync.c
- （2）broadcast_audio_sync.h
- （3）broadcast_dec.c
- （4）broadcast_dec.h
- （5）broadcast_enc.c
- （6）broadcast_enc.h
- （7）broadcast_codec.h

应用层相关

- （1）app_connected.c
- （2）app_connected.h
- （3）connected.c

- (4) `connected_api.h`
- (5) `cig_params.c`
- (6) `wireless_params.h`

2. 文件介绍

由于蓝牙相关文件改动较少，此处不做深入介绍。

应用层的代码主要由六个文件组成，还有部分代码穿插在其他文件的应用流程里面，此处重点介绍主要的六个文件。

(1) `app_connected.c`

该文件存放了应用流程二次开发所需的 API，BIG 连接等相关事件获取的 API。

(2) `app_connected.h`

该文件为为 `app_connected.c` 的头文件，存放了相关的结构体定义、函数声明。

(3) `connected.c`

该文件进行二次开发时改动较少。其代码内容主要是耦合蓝牙和音频相关接口，并把基础流程封成 API 给 `app_connected.c` 使用。

(4) `connected_api.h`

该文件为为 `connected.c` 的头文件，存放了相关的结构体定义、函数声明。

(5) `cig_params.c`

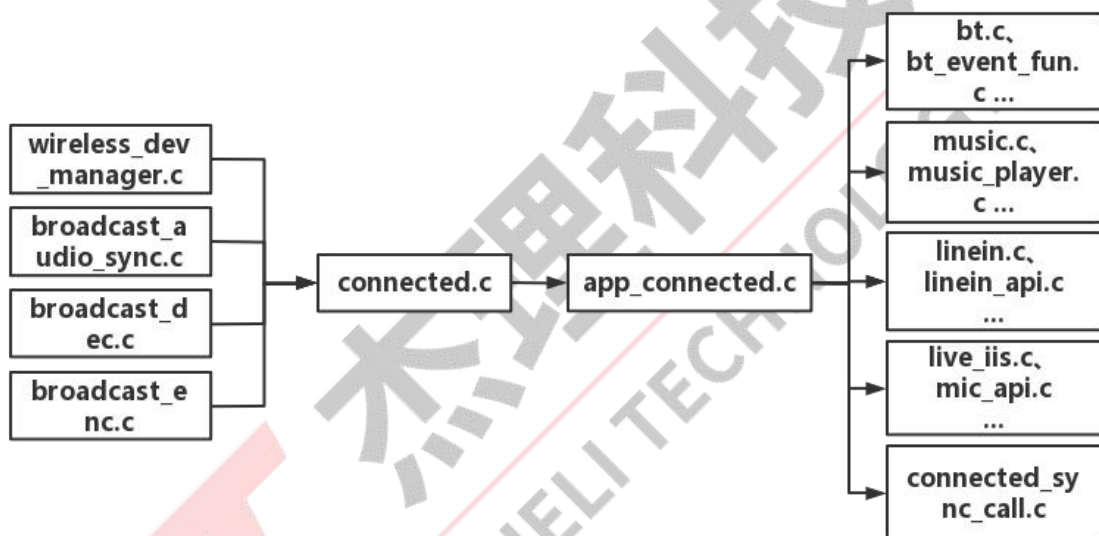
该文件主要存放 CIG 及音频编解码所需的参数以及设置参数的 API。每个模式对应一个参数表，在设置参数时会根据当前模式使用对应的参数表。

另外接收端音频编解码参数支持每次开启 CIG 时自动适配发送端的参数，保证发送端接收端参数一致。CIG 工作过程中暂不支持动态修改参数。

(6) wireless_params.h

该文件为为 `cig_params.c` 的头文件，存放了相关的结构体定义、函数声明。

3. 各个文件之间的调用关系



4. 启动和关闭流程

(1) CIG 整理开启流程与 BIG 类似。不同的地方是 CIG 的主机要和从机连接上才会返回连接成功事件(`CIG_EVENT_TRANSMITTER_CONNECT`), 这是由于 CIG 是具有连接动作的; 而 BIG 则是从机监听主机, 没有连接的概念, 因此 BIG 的主机是一开启就会返回连接成功事件。

(2) 带蓝牙后台情况下, 开机后应先进蓝牙模式, 再切换到其他模式, 因为进入蓝牙模式时会自动初始化蓝牙协议栈; 如果直接进入其他模式, 则

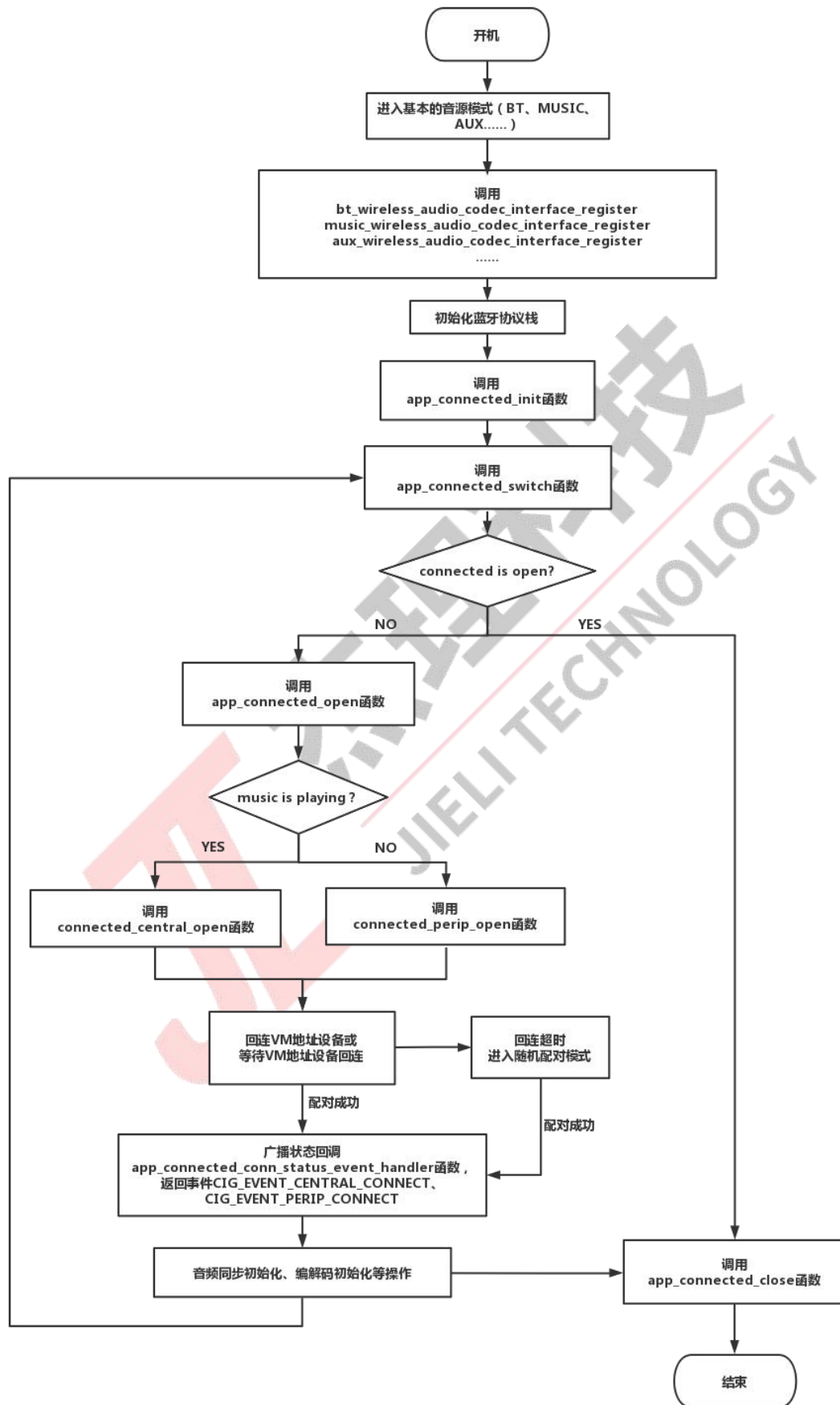
需手动初始化协议栈。

(3) 不带后台情况下，由于退出蓝牙模式时蓝牙协议栈也会被关闭，因此切换到其他模式后需手动初始化蓝牙协议栈，退出模式时也要手动关闭蓝牙协议栈。

(4) 手动初始化蓝牙协议栈可以调用 `btstack_init_in_other_mode` 函数，退出则调用 `btstack_exit_in_other_mode` 函数。

(5) 由于 CIG 连接时所运行的音频流程与非连接不一样，所以需要注册音频流程切换接口，可通过 `xxx_codec_interface_register` 来注册。

(6) 基于 CIG 的连接有地址配对和随机配对两种方式。SDK 默认 VM 有连接地址时优先进入地址配对，当地址配对超时后，设备会进入随机配对。只有主从处于相同配对模式时，才能配对上。例如，主设备在地址配对模式时配对从机地址，那么从机也要处于地址配对模式并配对主设备的地址，两者才能连接上；或者两者都处于随机配对时才能连接上。



(二) 参数及配置说明

1. board_jl701n_xxx_cfg.h

参数名/配置名	含义	默认值
TCFG_CONNECTED_ENABLE	CIG 总开关	0
LEA_CIG_CENTRAL_EN	CIG 主机开关	1
LEA_CIG_PERIPHERAL_EN	CIG 从机开关	1
CONNECTED_RECEIVER_CLOSE_EDR_CONN	CIG 从机关闭经典蓝牙可发现可连接	1
CONNECTED_KEY_EVENT_SYNC	主从按键事件同步开关	0
CONNECTED_TX_LOCAL_DEC_EN	CIG 音频发送端是否解码发送的音频	1
LEA_CIG_CONNECTION_NUM	配置连接设备数，最大配置为 2	1
JLA_CODING_SAMPLERATE	JLA 编解码采样率	48000（暂不支持 44100）
JLA_CODING_FRAME_LENGTH	一帧编码的 pcm 时间长,单位 ms	100（只支持 25, 50, 100）
JLA_CODING_CHANNEL	JLA 的通道数，其他参数不变，实际单声道的码率是双声道码率的 2 倍	2
JLA_CODING_BIT_RATE	JLA 的编解码码率	JLA_CODING_CHANNEL 为 1 时:64000 JLA_CODING_CHANNEL 为 2 时:96000
TCFG_LIVE_AUDIO_LOW_LATENCY_EN	实时音频低延时使能，BIG 和 CIG 两种模式均可使用	0

2. cig_params.c

参数名/配置名	含义	默认值
CIG_PAIR_NAME	配对名	"br28_cig_soundbox"
USED_CIS_NUM	BIS 发送通道数	1
XXX_SDU_PERIOD_MS	蓝牙发包时间间隔	10
cig_xxx_codec_params	不同音源模式对应的 JLA 编解码参数	见代码
cig_xxx_tx_param	不同音源模式对应的 CIG 发送参数。其中.rtn 为重复次数，.mtl 为预发包数量。rtn 和 mtl 都会影响接收端收包质量和距离，两个参数越大，CIG 距离越远，经典蓝牙距离越短。另外，增大 mtl 还是增大延时。aclMaxPduCToP 和.aclMaxPduPToC 分别为主机发给从机最大 ACL 数据长度和从机发给主机最大 ACL 数据长度，开发者可根据实际情况进行调整。	见代码
cig_rx_param	CIG 接收参数。其中部分参数含义与 cig_xxx_tx_param 中的相同。	见代码

3. connected.c

参数名/配置名	含义	默认值
CONNECTED_DEC_OUTPUT_CHANNEL	选择解码器解码输出的声道	AUDIO_CH_LR

4. 其他文件

参数名/配置名	含义	默认值
xxx_audio_process_switch_func	配置普通音源模式音频流程和 CIG 音频流程切换接口，本地音频播放和暂停状态获取接口	见代码
CIG_TRANSPORT_MODE	CIG 通讯方式配置	见 wireless_params.h

（三）ACL 数据交换

由于 CIG 是双向通讯协议，因此支持任务同步执行、按键消息同步和互发数据给对方。

1. 任务同步执行

位置	connected_sync_call.c (connected_api_sync_call_by_uuid 函数)
说明	发送命令给对方，双方通过 uuid 找到注册的回调函数，并约定一段时间后同时执行，具体可参考下面代码。
代码	<pre> 1. void main(void) 2. { 3. int data; 4. connected_api_sync_call_by_uuid('C', &data, 400, CONNECTED_DEVICE_IDENTIFICATION_L); 5. } 6. 7. static void common_state_sync_handler(int state, int sync_call_role) 8. { 9. switch (state) { 10. default: 11. g_printf("%s %d, state:0x%x, sync_call_role:%d", __FUNCTION__, __LINE__, state, sync_call_role); 12. break; 13. } 14. } 15. </pre>

	<pre> 16. CONNECTED_SYNC_CALL_REGISTER(common_state_sync) = { 17. .uuid = 'C', 18. .task_name = "app_core", 19. .func = common_state_sync_handler, 20. }; </pre>
--	--

2. 按键消息同步

位置	connected_sync_call.c (connected_key_event_sync 函数)
说明	按键消息流程已添加，开发人员只需打开 CONNECTED_KEY_EVENT_SYNC 宏就可以。
代码	

3. 互发数据给对方

位置	connected_sync_call.c (connected_send_data_to_sibling 函数)
说明	发送数据给对方，对方通过 uuid 在对应的回调函数接收数据。
代码	<pre> 1. void main(void) 2. { 3. int data; 4. connected_send_data_to_sibling('C', &data, sizeof(data), CONNECTED_DEVICE_IDENTIFICATION_L); 5. } 6. 7. static void data_transmit_handler(u8 trans_role, void *data, u8 len) 8. { 9. int trans_role = argv[0]; 10. u8 *data = (u8 *)argv[1]; 11. int len = argv[2]; 12. g_printf("%s %d, length:%d, trans_role:%d", __FUNCTION__, __LINE__, len, trans_role); 13. put_buf(data, len); 14. //由于转发流程中申请了内存，因此执行完毕后必须释放 </pre>

```

15.     if (data) {
16.         free(data);
17.     }
18. }
19.
20. CONN_DATA_TRANS_STUB_REGISTER(data_transmit) = {
21.     .uuid = 'C',
22.     /* 没有给 task_name 赋值时, func 默认在收数中断执行。当 func 在收数中断执行时,
        func 内的代码尽量简短, 避免对后续收数产生影响 */
23.     .task_name = "app_core",
24.     .func = data_transmit_handler,
25. };

```

四、音频流程介绍

（一）流程介绍

BIG 和 CIG 的音频流程与音源模式的普通音频流程有所不同。BIG 和 CIG 的音频数据需要进行再编再解的过程，因此在普通音频模式下开关 BIG 或 CIG 会有一个音频流程切换的动作。在音频流程切换的过程中会调用结构体 `wireless_xxx_audio_ops` 中的函数指针，其中 `capture` 为 BIG 和 CIG 所用的音频流程开关接口，`play` 为普通音频流程开关接口。`capture` 和 `play` 的接口分别在 `live_audio_capture.c` 和 `live_audio_player.c` 中调用。

（二）BIG 和 CIG 音频流程算法接入

在 `capture` 中音频数据是根据函数 `live_audio_capture_open` 中模块初始化的顺序进行处理的。函数 `live_audio_capture_open` 中每个模块初始化时候会把对应的节点添加到链表中，当需要执行的时候会按照添加链表的顺序执行。其中有一个 `effect` 的模块为音效模块，里面主要对音频数据进行算法的处理，开发者可在这个模块的执行函数 `live_audio_effect_write_frame` 中添加自己

的处理。在使用 `effect` 模块时需要手动打开文件 `live_audio_capture.c` 中的宏 `LIVE_AUDIO_CAPTURE_EFFECT_ENABLE`。

五、常见问题及解决方法

（一）发送 **ACL** 数据异常

1. Note: cis perip acl tx hdl(0x50) > 9

异常原因：发送的 **ACL** 数据超过默认的长度限制。

解决方法：修改 `cig_params.c` 中对应所使用的结构体变量 `.aclMaxPduPToC`。

